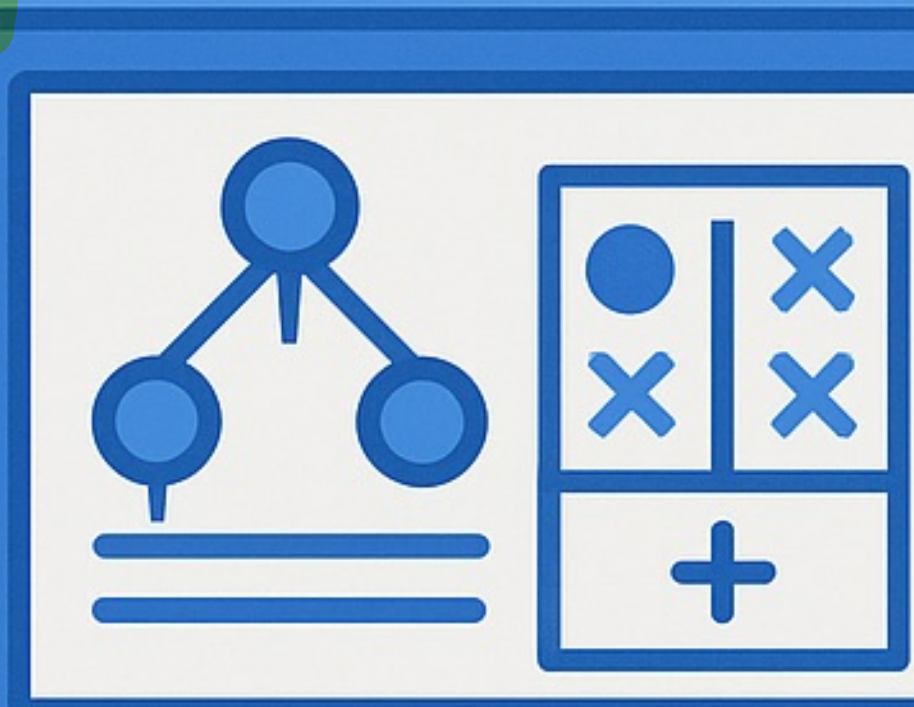
MACHINE LEARNING

SUPERVISED LEARNING

Class start @9:05pm





Lecture 3: Decision Tree 1

#Agenda

1) KNN Imputation ✓
2) Imbalanced data
3) Decision Tree

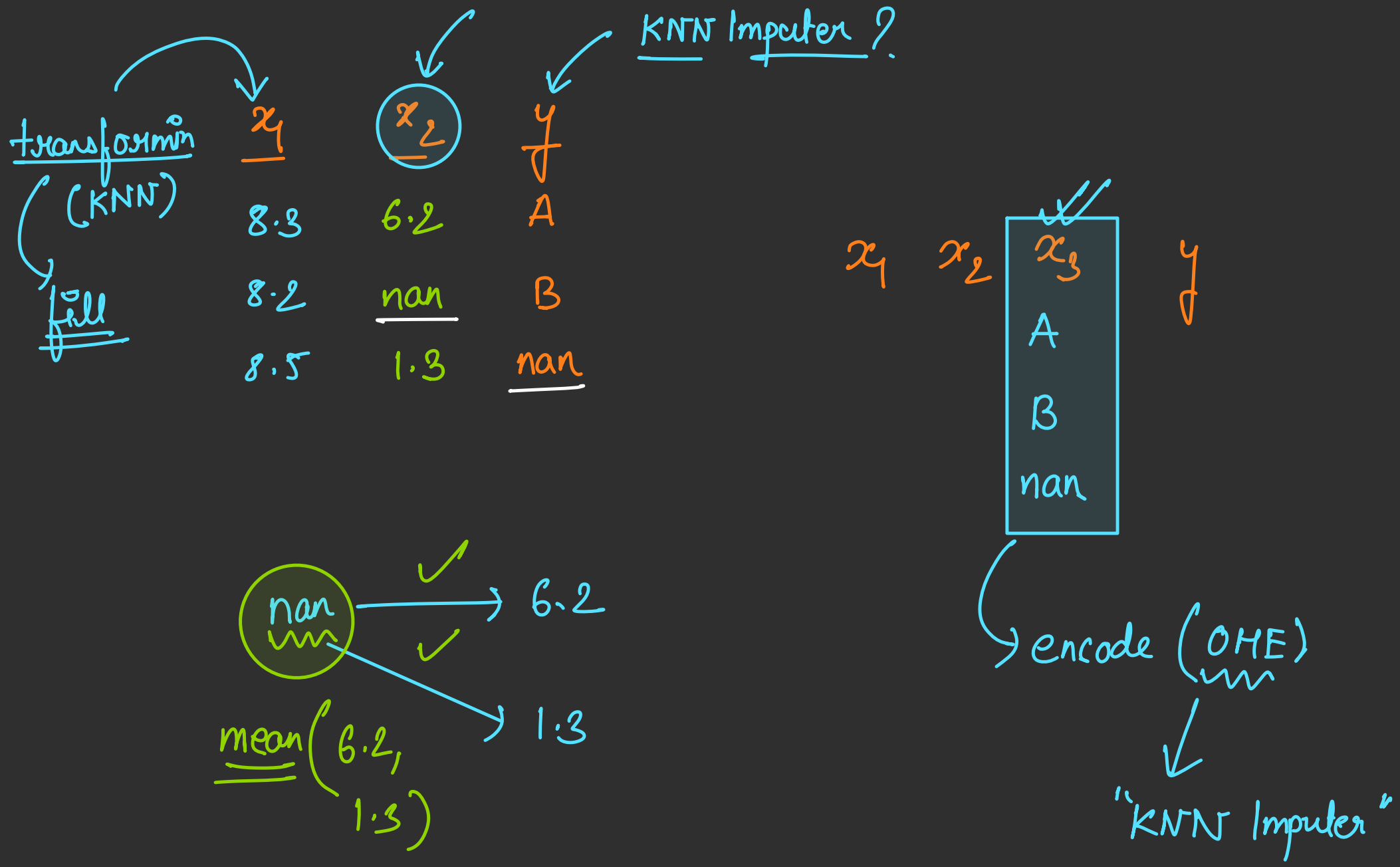
KNN Imputation



It helps to impute missing value based on its neighbouring values

Avg $(x_4, x_2, x_6, x_7, x_{10}, x_{20})$

filled for the white dot


KNN Imputer ?
transformin
(KNN)
fill
x1
x2
y
8.3
6.2
A
8.2
nan
B
8.5
1.3
nan
nan
6.2
1.3
mean(6.2, 1.3)
x1
x2
x3
y
A
B
nan
encode (OHE)
"KNN Imputer"

StandardScaler()

$$z = \frac{x - \mu}{\sigma}$$




X_{train}
X_{test}
fit_transform
transform
(μ, σ) → Z value

Imbalanced data

Class 1 ⟶ 500
Class 0 ⟶ 280
} Imbalanced

Is KNN affected by Imbalanced data?

→ It will create a problem in may majority vote calc. Hence may always result in answering the dominating class.

Two way you can solve this

1) Under sampling

2) Over Sampling

Undersampling

↳ You reduce down the majority class.

Class 1 → 500
Class 0 → 280
} Imbalanced ⇒
Class 1 → 280
Class 0 → 280
} Balanced

Easy way but also somewhat problematic

W/ Cons: You lose out on data pt / information

If you have large amount of data, undersampling is ok

Over sampling (It creates new records for the minority class)

1) Up sampling → "Randomly select some data from the min class and add them to the min class"

2) SMOTE

30 records → RS (10) → 40 records

Min

"Variance reduction" ↓

Up sampling is done when there is very few % of Imbalanced data and the dataset is large enough.

{ Class 1 → 58000
  Class 0 → 49000 } ✓✓

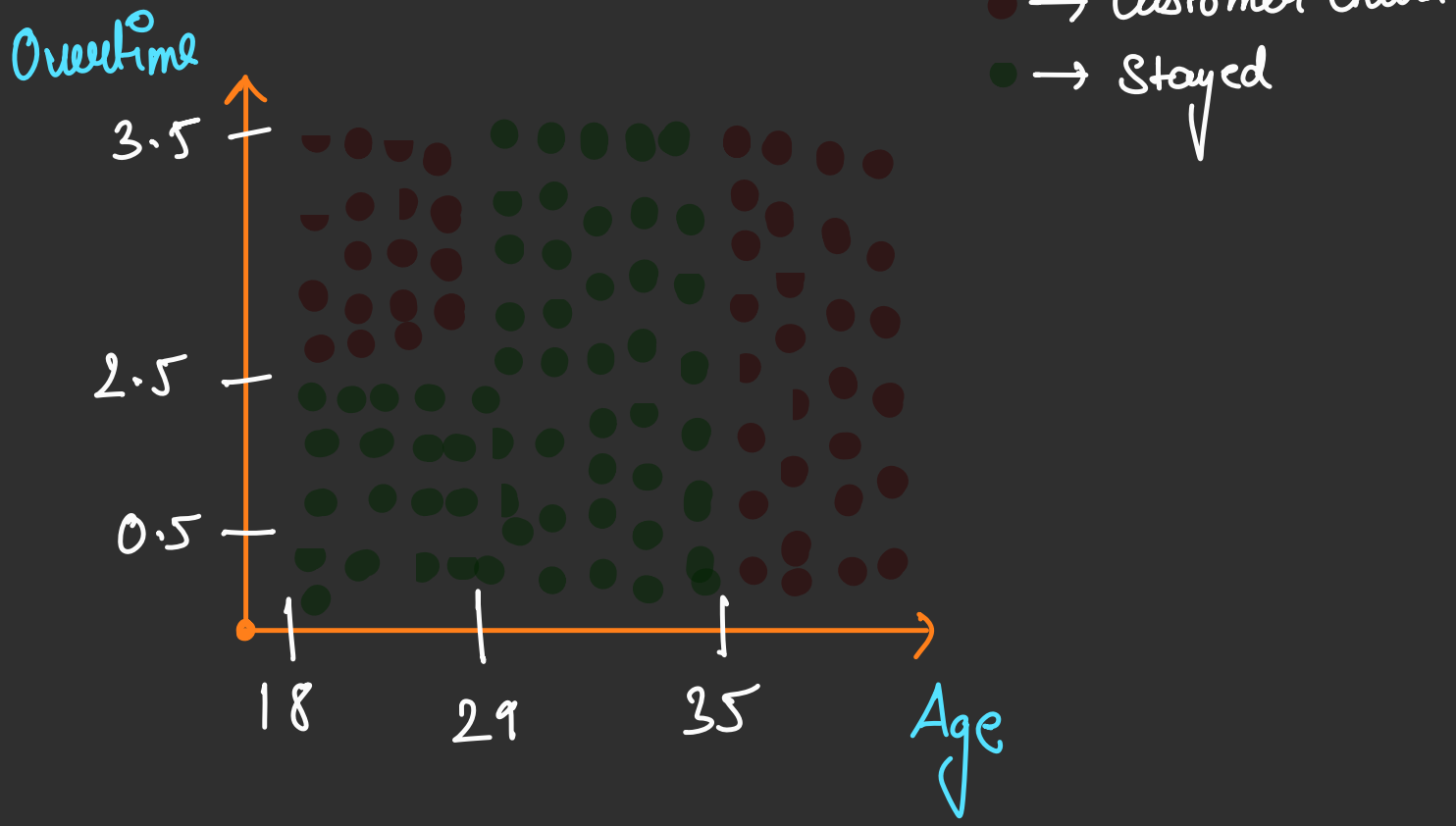
SMOTE

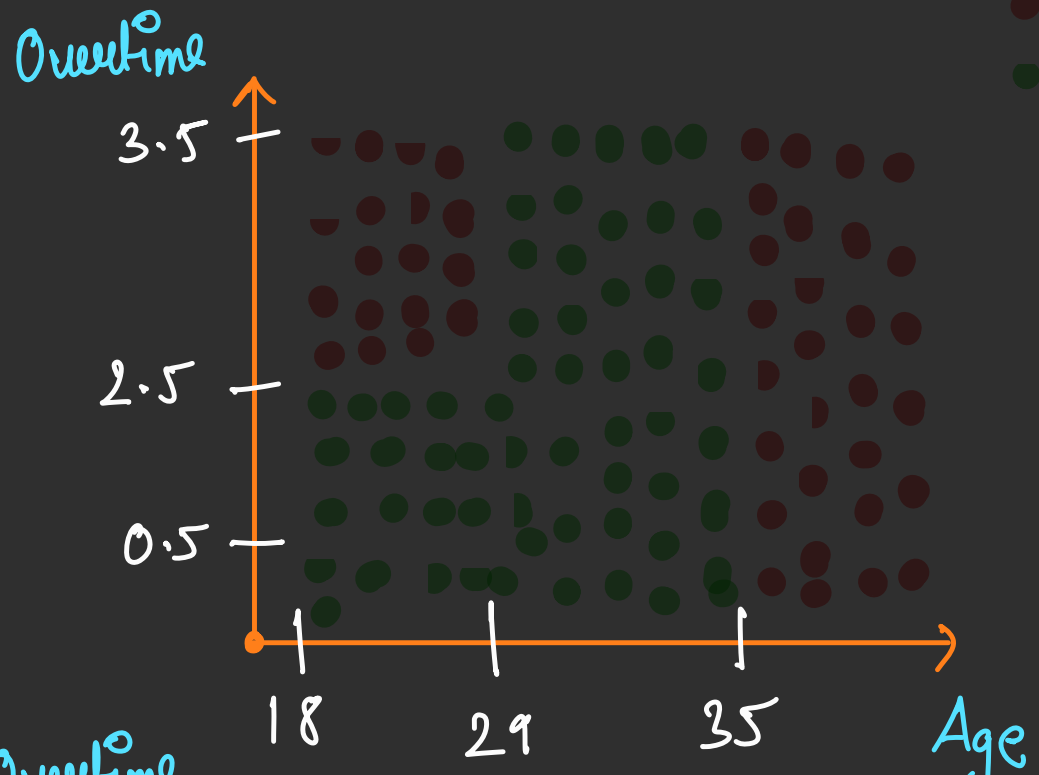
↳ Create synthetic data from your existing dataset

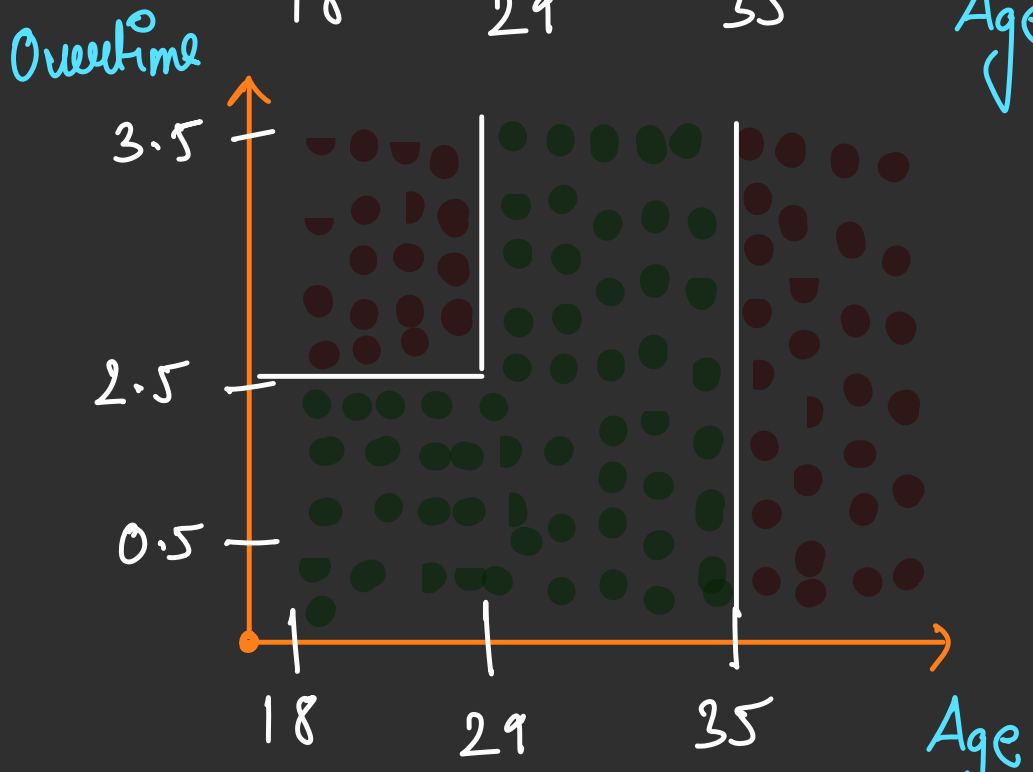
x_1	x_2	x_3	Target
A	2	1.5	0
B	3	6.2	0
C	5	1.8	0
B	2	1.8	0

Decision Tree

35 features ⟶ Age Vs Overtime





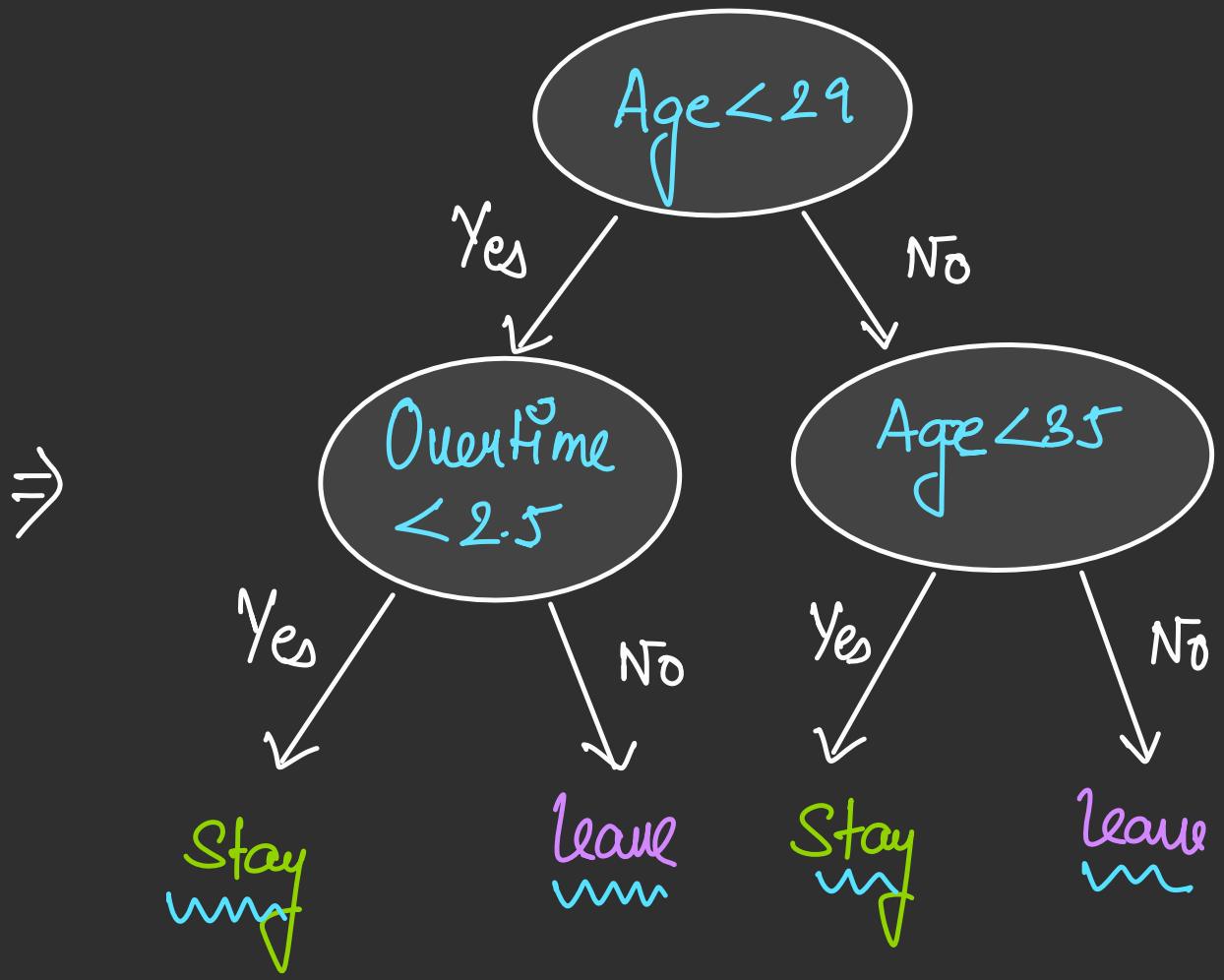


● → Customer Churn

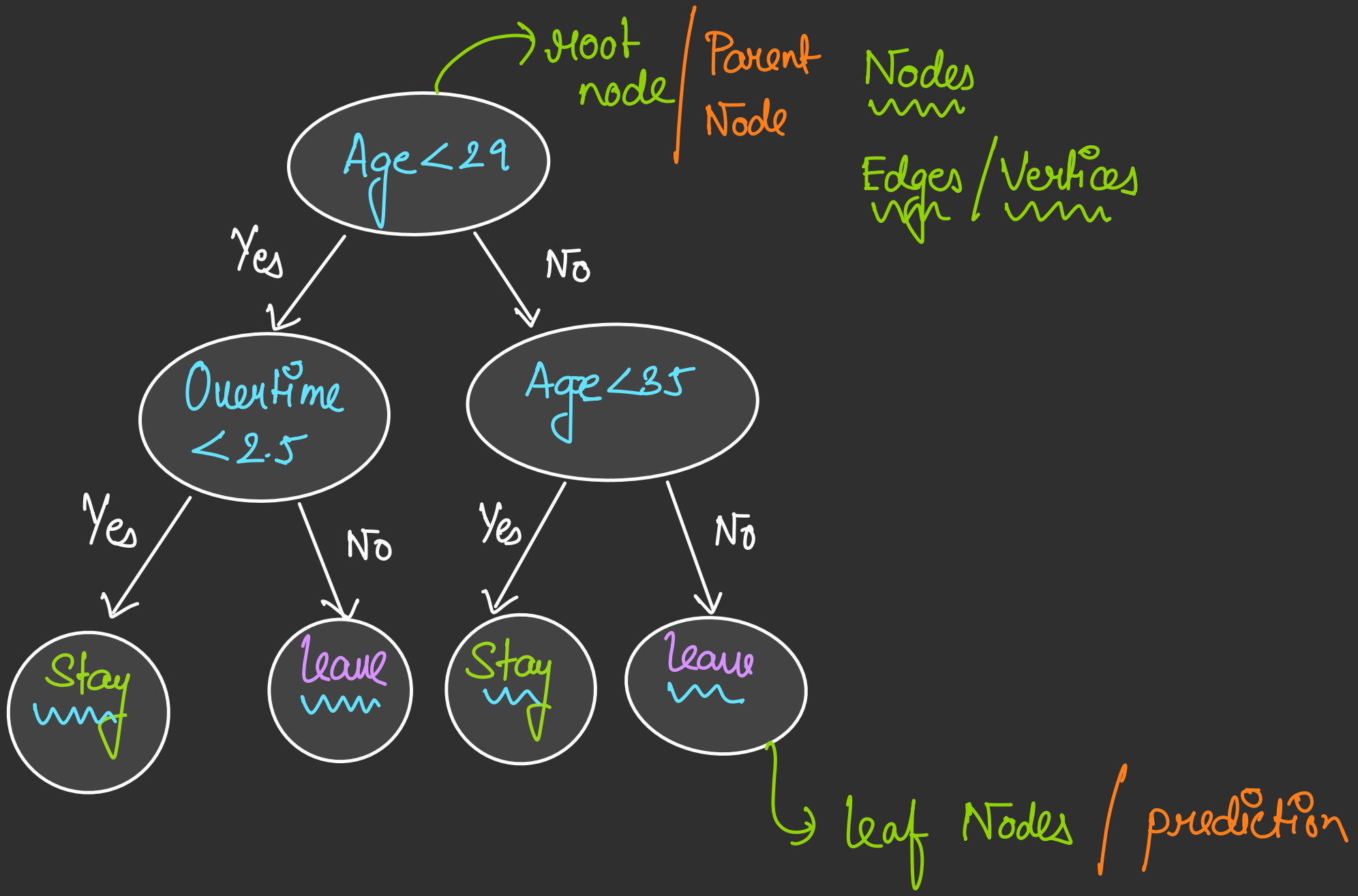
● → Stayed

```
If Age < 29:
    if overtime < 2.5
        Stay
    else:
        leave
else:
    if Age < 35
        Stay
    else:
        leave
```

```
if Age < 29:
    if overtime < 2.5
        Stay
    else:
        leave
else:
    if Age < 35
        Stay
    else:
        leave
```

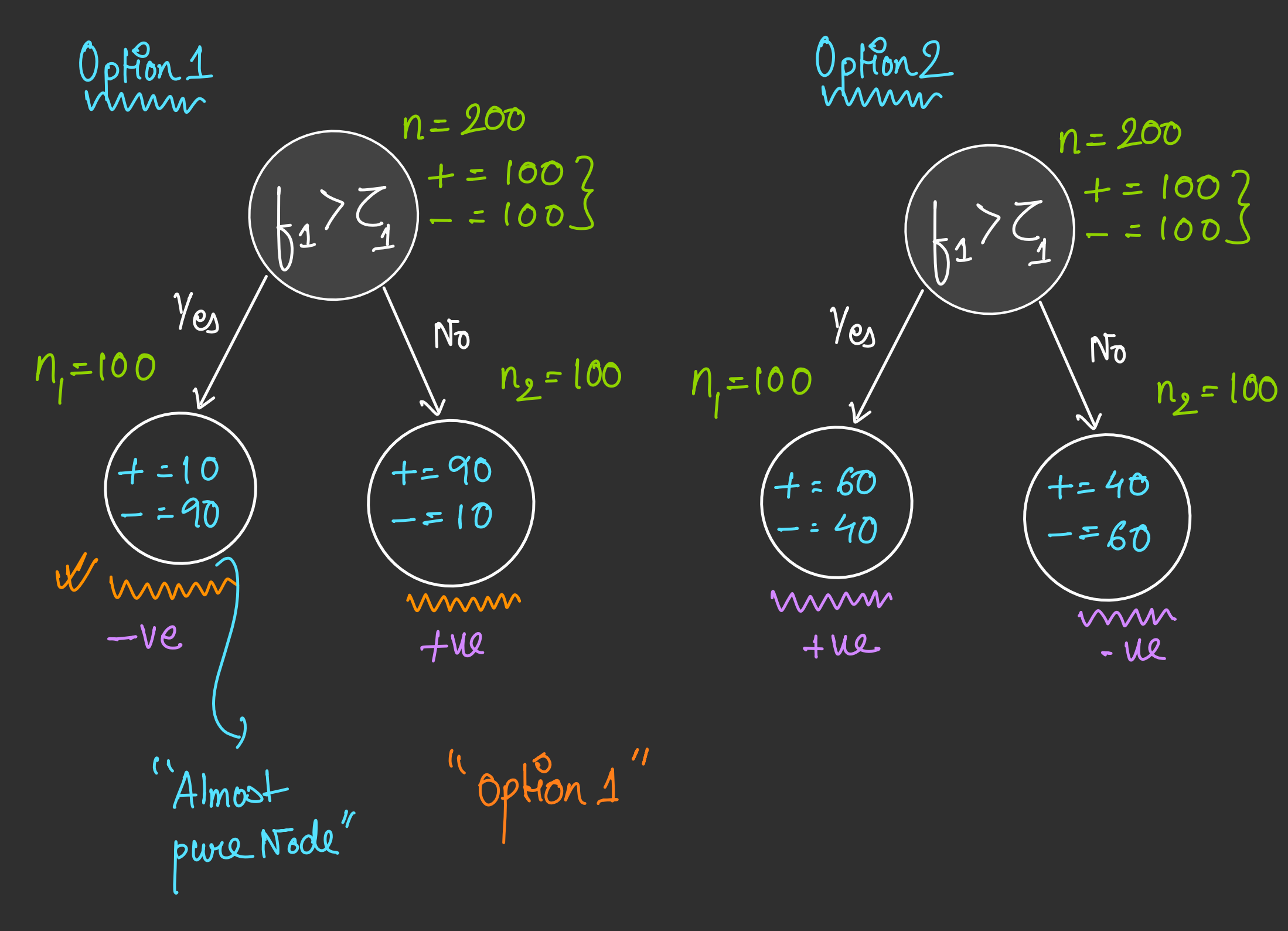


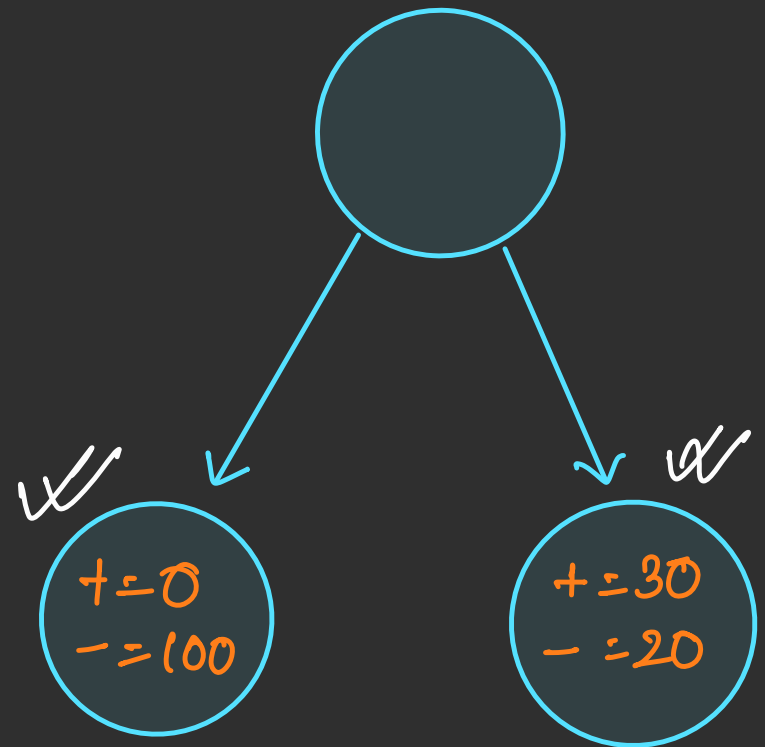
"Decision Tree"


Root node / Parent Node
Nodes
Edges / Vertices
Age<29
Yes
No
Overtime <2.5
Age<35
Yes
No
Yes
No
Stay
Leave
Stay
Leave
Leaf Nodes / prediction

"How do I construct this tree?"

"DT always have axis parallel Decision Boundary"


Option 1
$f_1 > \tau_1$
n = 200
+ = 100
− = 100
Yes
No
$n_1 = 100$
$n_2 = 100$
+ = 10
− = 90
+ = 90
− = 10
−ve
+ve
"Almost pure Node"
"Option 1"
Option 2
$f_1 > \tau_1$
n = 200
+ = 100
− = 100
Yes
No
$n_1 = 100$
$n_2 = 100$
+ = 60
− = 40
+ = 40
− = 60
+ve
-ve



Pure Node

We will find the pure Node

"pure Node means more Homogeneous data"

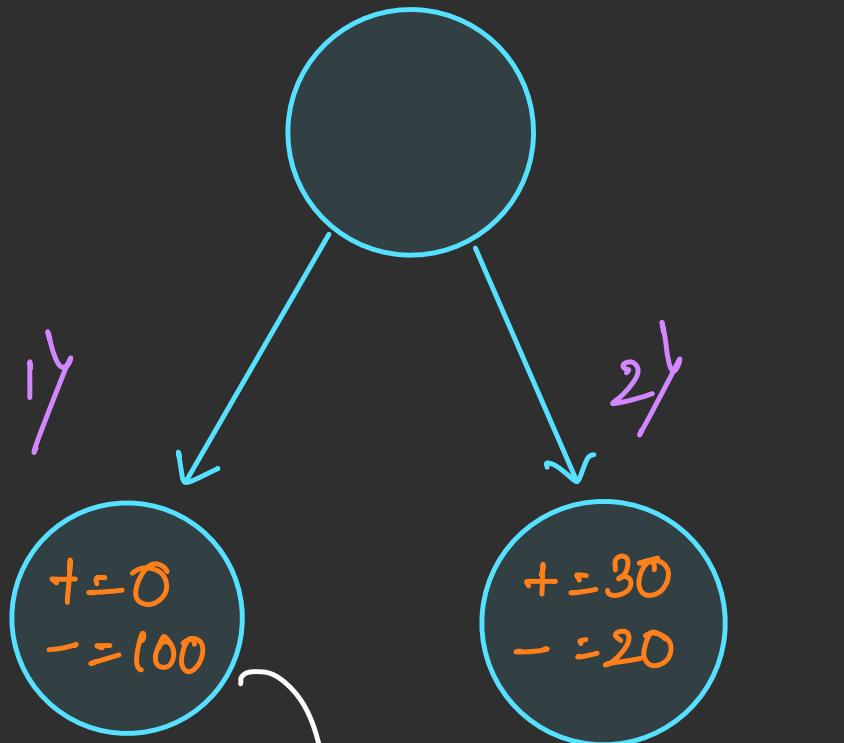
(Data only belonging to one class)

Pure / Impure

↳ Entropy (Randomness in your system)

1) ↑ Entropy → ↑ Impurity → ↑ heterogenous → ↓ homogenous

2) ↓ Entropy → ↓ Impurity → ↓ heterogenous → ↑ homogenous

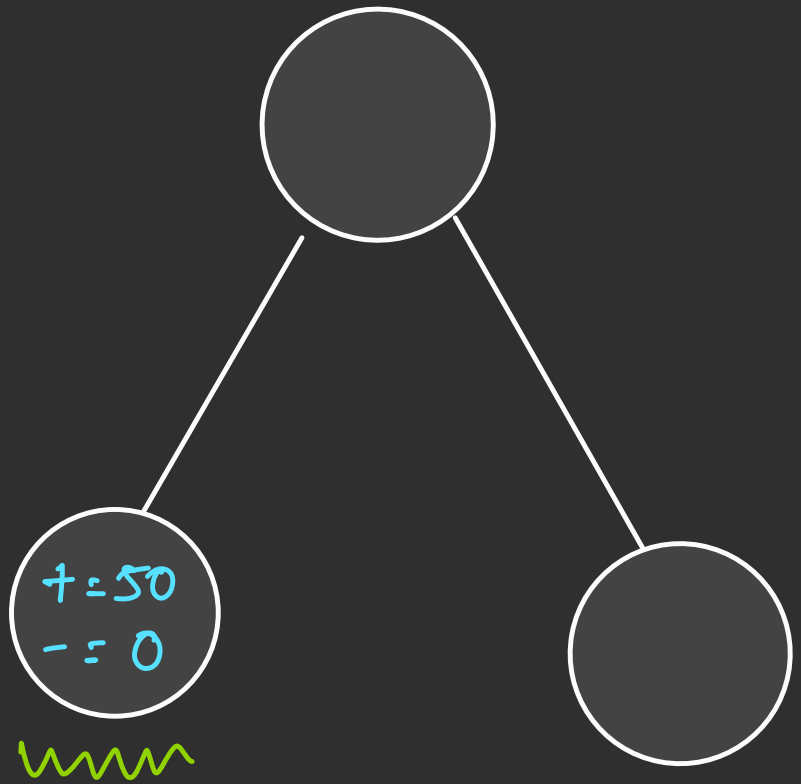


Entropy ↓

Entropy ↑

→ homogenous = all the dpt belong to 1 clas

Quiz



Homogeous / Pure Node

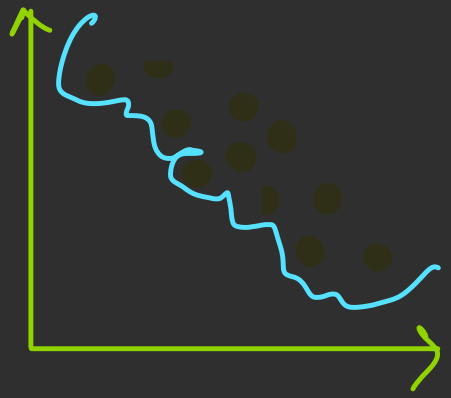
"Entropy = 0"

Doubts

Overfit = Low Bias + High Variance

↳ Perfect predictio

↳ Model has even learnt all the edge cases



Memorized all training data

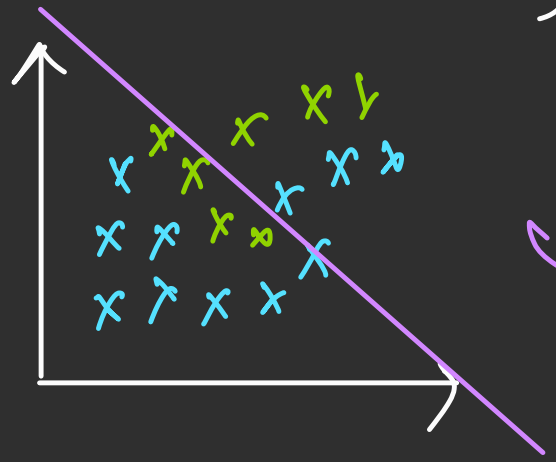
Train ↑ Test ↓

overfit model

Underfit = High Bias + Low Variance

Lot of misclassification

Too general model



Model didn't learn anything

Train ↓ Test ↓

Underfit model

logistic Regression ($C = 1/\lambda$)

grid serach CV ($C = [1, 0.1, 0.5]$)

→ different model training on diff subset of data

C is best

class 1 = 100
class 0 = 20



class 1 ⇒ 100 × 0.20

class 0 ⇒